

UNIVERSITY OF PENNSYLVANIA
ESE 650: LEARNING IN ROBOTICS
HOMEWORK 4
DUE: 05/01 11.59 PM ET

Read the following instructions carefully before beginning to work on the homework.

- You will submit neatly written solutions via Gradescope. You can use $\text{\LaTeX}$ (encouraged) but you can also write by hand. You can use `hw_template.tex` on Canvas in the “hw” folder to do so. If your handwriting is *unambiguously legible*, you can submit PDF scans/tablet-created PDFs.
- Please start a new problem on a fresh page and mark all the pages corresponding to each problem. Failure to do so may result in your work not graded completely.
- Clearly indicate the name and Penn email ID on your submitted solutions.
- For each problem in the homework, you should mention the total amount of time you spent on it.
- You can be informal while typesetting the solutions, e.g., if you want to draw a picture feel free to draw it on paper clearly, click a picture and include it in your solution. Do not spend undue time on typesetting solutions.
- You will see an entry of the form “HW 0 PDF” where you will upload the PDF of your solutions. You will also see entries like “HW 0 Problem 1 Code” where you will upload your solution for the respective problems. **For each programming problem, you should create a fresh Python file.** This file should contain all the code to reproduce the results of the problem and you will upload the .py file to Gradescope. If we have installed Autograder for a particular problem, you will use the Autograder. Name your file to be “pennkey_hw0_problem1.py”, e.g., I will name my code for Problem 1 as “pratikac_hw0_problem1.py”.
- **You should include all the relevant plots in the PDF, without doing so you will not get full credit.** You can, for instance, export your Jupyter notebook as a PDF (you can also use text cells to write your solutions) and export the same notebook as a Python file to upload your code.
- **Your PDF solutions should be completely self-contained. If the question requires you to produce a plot, you must have it in the PDF to get credit.**

We will run the Python file to check if your solution reproduces the results in the PDF.

Credit. The points for the problems add up to 100. You only need to solve for 100 points to get full credit, i.e., your final score will be $\min(\text{your total points}, 100)$.

1 **Problem 1 (Mujoco Model Predictive Control Library, 25 points (No Auto-**
2 **grader)).** In this problem, we will learn to use the Mujoco MPC (MJPC) library
3 (https://github.com/google-deepmind/mujoco_mpc). MJPC allows the user to solve
4 surprisingly complex robotics tasks using non-learning based planners, e.g., cross-
5 entropy method, or policy gradients where one samples a lot of trajectories and
6 simply executes the one that obtains the best return in a receding horizon fashion.

7 **(a) [10 points]** Use the Installation instructions on Github in the Readme to install
8 mjpc. If you have a Linux or Mac laptop, this should be easy. Windows is also
9 supposed to work according to the original authors. But it would be best to do this
10 problem on a friend's non-Windows laptop if you can. That way you can avoid fighting
11 with Windows. You should read the paper at <https://arxiv.org/abs/2212.00541>. You
12 should also listen to this seminar by Yuval Tassa who is one of the main authors
13 <https://www.youtube.com/watch?v=J-JO-lgaKtw>. You do not need VS Code like
14 the Readme says. You can simply do

```
15 $ git clone https://github.com/google-deepmind/mujoco_mpc.git
16 $ cd mujoco_mpc
17 $ mkdir build; cd build;
18 $ cmake ..
19 $ make -j
```

20 After this, you should have a file “build/bin/mjpc” which is the program you will
21 run as “./mjpc” from your terminal.

22 **(b) [10 points]** The goal of this problem is to see a lot of different dynamical systems
23 in action and appreciate the power of fast rollouts in the simulators. You should
24 watch the video that we have recorded at
25 https://drive.google.com/file/d/1gc9nc5LgSYeVYMSDUhS7Y9iHkXP9OwY1/view?usp=drive_link
26 that describes the different parts of mjpc and how to use the widgets. There are a lot
27 of intricate and cool aspects to the different algorithms implemented in MJPC and I
28 would strongly encourage you to play with this program.

29 For Cartpole, Acrobot and the Walker, you will create a video just like the video we
30 created above (Zoom screen capture works great...). A sequence of screen shots that
31 convinces us that you were able to run the program and control these three systems
32 is also okay.

33 **(c) [5 points]** There is a system named “Humanoid Stand” in MJPC. Try to make
34 the humanoid stand up. You are free to modify any setting within MJPC.

1 **Problem 2 (PPO for a Walker, 75 points (No Autograder))**. In this problem, you
2 will use make the Walker walk using reinforcement learning. We will implement
3 PPO.

4 **(a) [5 points]** We will be using Deepmind’s Control Suite to implement our algo-
5 rithms. You should install this from https://github.com/google-deepmind/dm_control;
6 you can simply do “pip install dm_control”. You might find it useful to glance at
7 <https://arxiv.org/abs/2006.12983> to understand what the broad goals of the library
8 are. DM Control is a software package that wraps around the Mujoco simulator (not
9 MJPC). This is useful to implement a number of control and reinforcement learning
10 algorithms.

11 We have provided you some starter code to set up the environment. For example,
12 the few lines below initialize the Walker, create a task (i.e., a problem) called “walk”
13 (which tells DM Control to give us rewards that correspond to good walking) and
14 uses a random controller to undertake actions. You can run the code to see the
15 Walker in action inside a visualizer.

```
60 """
61 Setup walker environment
62 """
63 r0 = np.random.RandomState(42)
64 e = suite.load('walker', 'walk',
65               task_kwargs={'random': r0})
66 U=e.action_spec();udim=U.shape[0];
67 X=e.observation_spec();xdim=14+1+9;
68
69
70 #Visualize a random controller
71 def u(dt):
72     return np.random.uniform(low=U.minimum,
73                               high=U.maximum,
74                               size=U.shape)
75 viewer.launch(e,policy=u)
```

17 You will next implement rollouts using a neural network-based controller. See the
18 code below to understand how one should sample trajectories from the environment.

```
35 def rollout(e,uth,T=1000):
36     """
37     e: environment
38     uth: controller
39     T: time-steps
40     """
41
42     traj=[]
43     t=e.reset()
44     x=t.observation
45     x=np.array(x['orientations']).tolist()+[x['height']]+x['velocity'].tolist()
46     for _ in range(T):
47         with th.no_grad():
48             u,_=uth(th.from_numpy(x).float().unsqueeze(0))
49             r = e.step(u.numpy())
50             x=r.observation
51             xp=np.array(x['orientations']).tolist()+[x['height']]+x['velocity'].tolist()
52
53             t=dict(xp=xp,r=r.reward,u=u,d=r.last())
54             traj.append(t)
55             x=xp
56             if r.last():
57                 break
58     return traj
```

1 Line 48 samples the control u from a PyTorch-based neural network and runs the
 2 simulator one step forward by calling `e.step(u)`. This returns a data structure that
 3 contains the state (in this case, the orientations of different joints, height of the center
 4 of mass, and velocities of joints). It is a 24 dimensional state. The simulator also
 5 returns, the reward and whether this was the last state or not (`r.last()`). We collect all
 6 of returns from successive time steps for implementing the RL algorithm.

7 Your neural network-based controller should take in all 24 states as input and return 6
 8 controls as the output; check above that `udim = 6`. We should also build a stochastic
 9 controller, so the network will output a 6-dimensional mean and a 6-dimensional
 10 standard deviation for a Gaussian controller.

11 **(b) [50 points]** It might seem difficult to implement PPO but it is actually quite easy if
 12 you proceed systematically. You can refer to the implementation of PPO in Spinning
 13 Up <https://github.com/openai/spinningup/tree/master/spinup/algos/pytorch/ppo> as
 14 you begin to work on the problem. This implementation is less than 200 lines of
 15 code (if we ignore the boilerplate). You should also read the description of the
 16 algorithm in
 17 <https://spinningup.openai.com/en/latest/algorithms/ppo.html>.

18 **Important:** Spinning Up contains some of the best implementations of RL algo-
 19 rithms. But it is quite old and unmaintained now and therefore you will not be able
 20 to execute the code. It also uses parallelization to sample more trajectories but we do
 21 not need to do so. Use the Spinning Up implementation as a resource to understand
 22 how to implement the algorithm, do not simply code from there and expect it to
 23 work. We are using DM Control while Spinning Up is using Open AI Gym; the
 24 two are very different frameworks, different rewards, slightly different systems, and
 25 certainly syntax etc.

26 There is a minor difference in how we taught PPO in the lecture notes and how it is
 27 implemented here. Notice that at each iteration we want to calculate the gradient of

$$\mathbb{E}_{x \sim d^{\theta'}} \mathbb{E}_{u \sim u^{\theta'}} \left[\frac{u_{\theta}(u | x)}{u_{\theta'}(u | x)} a^{\theta'}(x, u) \right] - \lambda \mathbb{E}_{x \sim d^{\theta'}} [\text{KL}(u_{\theta}(\cdot | x), u_{\theta'}(\cdot | x))]$$

28 where the expectation is taken the states and controls along the trajectories in the
 29 rollout. In principle, we should fit a value function $q^{\theta'}(x, u)$ and a value function
 30 $v^{\theta'}(x)$ to calculate the advantage. But Spinning Up uses a shortcut and sets $q^{\theta'}(x, u)$
 31 to be the return of the trajectory that was rolled out starting from state x and control
 32 u . This will be a poor estimate of q but it is certainly valid—and it is quite simple to
 33 implement. It also normalizes the returns to come of all the states by subtracting the
 34 mean returns of all states in the rollout and dividing by the standard deviation; see
 35 Line 81. They fit a critic $v^{\theta'}(x)$ using the Bellman iteration on Line 248.

36 A suggested order for implementing things:

- 1 (1) Implement the neural networks for the controller and the value function.
2 Be careful of conversions from PyTorch to Numpy and dimensions of all
3 quantities (controller takes in a mini-batch of states, outputs a mini-batch
4 of controls along with their log-likelihoods, the value function takes in
5 a mini-batch of states as input and outputs a mini-batch of values). It is
6 important to appreciate that we do need the gradient of the log-likelihood of
7 the control in PPO, so you need to use the PyTorch routines to sample the .
8 See lines 127-132 in core.py for how the control is sampled with `th.no_grad`
9 and Line 231 for obtaining the controls with the log-likelihood.
- 10 (2) Code up the loss function for PPO correctly. Check whether the gradient
11 computed using backward is non-trivial (not NaNs, correct dimensions etc.).
- 12 (3) You should train for at least 1 million time-steps. Since the Walker is going
13 to fall down quickly at the beginning, each episode will be short. As the
14 controller learns, the episodes become longer (the code given to you has a
15 maximum length of 1000). So 1 million timesteps amounts to approx. 500
16 rollouts.
- 17 (4) Describe in detail your implementation and any changes/variations that you
18 made in the PDF.
- 19 **(b) [20 points]** Plot the average return of the trajectories sampled during rollout as a
20 function of the number of timesteps. See an example plot at
21 <https://spinningup.openai.com/en/latest/spinningup/bench.html>. Your numerical
22 values of the returns will be different because DM Control has a different reward
23 than Open AI's Gym. You should include screenshots of the Walker walking in the
24 DM Control viewer in the PDF.

If you want to first attempt a simpler problem than walking, you can do the task named "stand" for the Walker.